Real Time Pipeline - Camera to GPU to Display

Earl Wong

Overview

- You capture 4K, 8K video … at 60, 120 fps … process the frames in the GPU … and output the frames to the display.
- How is this accomplished in real time?
- We will use Android, C, Kotlin, and OpenGL to help illustrate the path.

Overview

- After the sensor input is processed by the ISP (assuming non-RAW output), the frames are written to DRAM.

- On SoC's, the DRAM is shared by the CPU, GPU, NPU, Display, and Encoder.

- Accessibility and share-ability of the DRAM is determined by the classification:

1) Regular Virtual Memory

2) DMA Backed Memory

3) GPU Private Memory

Types of Memory Classifications

Regular Virtual Memory (CPU Owned)

- Regular Virtual Memory is intended for CPU use.
- Examples: byte[ ], ByteBuffer.allocate( ), malloc( ), new( )
- Regular Virtual Memory can be accessed by the GPU by copying the memory into GPU Private Memory.

Types of Memory Classifications

GPU Private Memory

- GPU Private Memory is allocated, managed, and owned by the GPU driver.
- This memory is optimized for GPU access.
- This memory is NOT directly shareable with other hardware blocks like the ISP, Display, or Encoder.
- Examples: glTexImage2D( ), glTexSubImage2D( ), and glBufferData( )

Virtual and Private Memory

- The video frames are brought into the GPU using Regular Virtual Memory and GPU Private Memory.

- We provide two sample code examples.

Simple Sample Code Example

```
uint8_t* pixels = malloc(width * height * 4);

glTexImage2D(GL_TEXTURE_2D,
0,
GL_RGBA,
width,
height,
0,
GL_RGBA,
GL_UNSIGNED_BYTE,
pixels);
```

Cost

- A copy occurs “under the hood” by glTexImage2D( ) to transfer the data from Regular Virtual Memory to GPU Private Memory.

- 4K video at 60 fps = 3840x2160x4x60 = 1.9GByte / sec

Detailed Sample Code Example

```
ImageReader reader = ImageReader.newInstance(width, height, ImageFormat.YUV_420_888, 2);

cameraCaptureSession.setRepeatingRequest(requestBuilder.build(), null, handler);

reader.setOnImageAvailableListener(listener, handler);

//Allocate once and reuse (avoid per-frame allocations!)
byte[] tightY  = new byte[width * height];
//byte[] tightUV = new byte[width * height / 2]; // NV12 interleaved UV
```

.....

```
// In ImageReader callback thread:
Image image = reader.acquireLatestImage();
if (image == null) return;

Image.Plane[] planes = image.getPlanes();
Image.Plane yP = planes[0];
Image.Plane uP = planes[1];
Image.Plane vP = planes[2];

//TWO FULL FRAME COPIES TO CPU
copyPlaneTight(yP, width, height, tightY);
interleaveUV_NV12(uP, vP, width, height, tightUV);

image.close();
```

Detailed Sample Code Example

```
//Go to GL thread and upload:
ByteBuffer yBB  = ByteBuffer.wrap(tightY);
ByteBuffer uvBB = ByteBuffer.wrap(tightUV);

GLES30.glPixelStorei(GLES30.GL_UNPACK_ALIGNMENT, 1);

//Upload Y and UV
GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, texY);
//COPY FROM CPU HEAP TO GPU MEMORY BY GPU DRIVER INTO GPU TEXTURE MEMORY
GLES30.glTexSubImage2D(GLES30.GL_TEXTURE_2D, 0, 0, 0, width, height,
GLES30.GL_RED, GLES30.GL_UNSIGNED_BYTE, yBB);

GLES30.glBindTexture(GLES30.GL_TEXTURE_2D, texUV);
//COPY FROM CPU HEAP TO GPU MEMORY BY GPU DRIVER INTO GPU TEXTURE MEMORY
GLES30.glTexSubImage2D(GLES30.GL_TEXTURE_2D, 0, 0, 0, width/2, height/2,
GLES30.GL_RG, GLES30.GL_UNSIGNED_BYTE, uvBB);
```

Detailed Sample Code Example

```
//GLSL
uniform sampler2D texY;
uniform sampler2D texUV; // RG = (U,V)
in vec2 vTex;
out vec4 outColor;

void main() {
float y = texture(texY, vTex).r;
vec2 uv = texture(texUV, vTex).rg;
:
:
```

Cost

- 12MBytes / frame x 4 copies = 48MBytes / frame

- 48MBytes / frame x 30fps = 1.440Gbytes/sec of memory bandwidth

The Cost of the “Copy”

- Both examples require data to be copied.
- Copy is BAD, BAD, BAD.
- Copy = more memory, more bandwidth, more power, more latency, more cache issues …
- = more of EVERYTHING that you do NOT want in a resource constrained environment.
- Next, we describe the workaround.

Types of Memory Classifications

Direct Memory Access (DMA) Backed Memory

- gralloc( ): DMA-BUF heap allocation
- Accessible by ISP, GPU, Display, Encoder hardware blocks
- CPU usage is possible, with correct access / usage flags.
- AHardwareBuffer( ): Native API wrapper around gralloc

gralloc

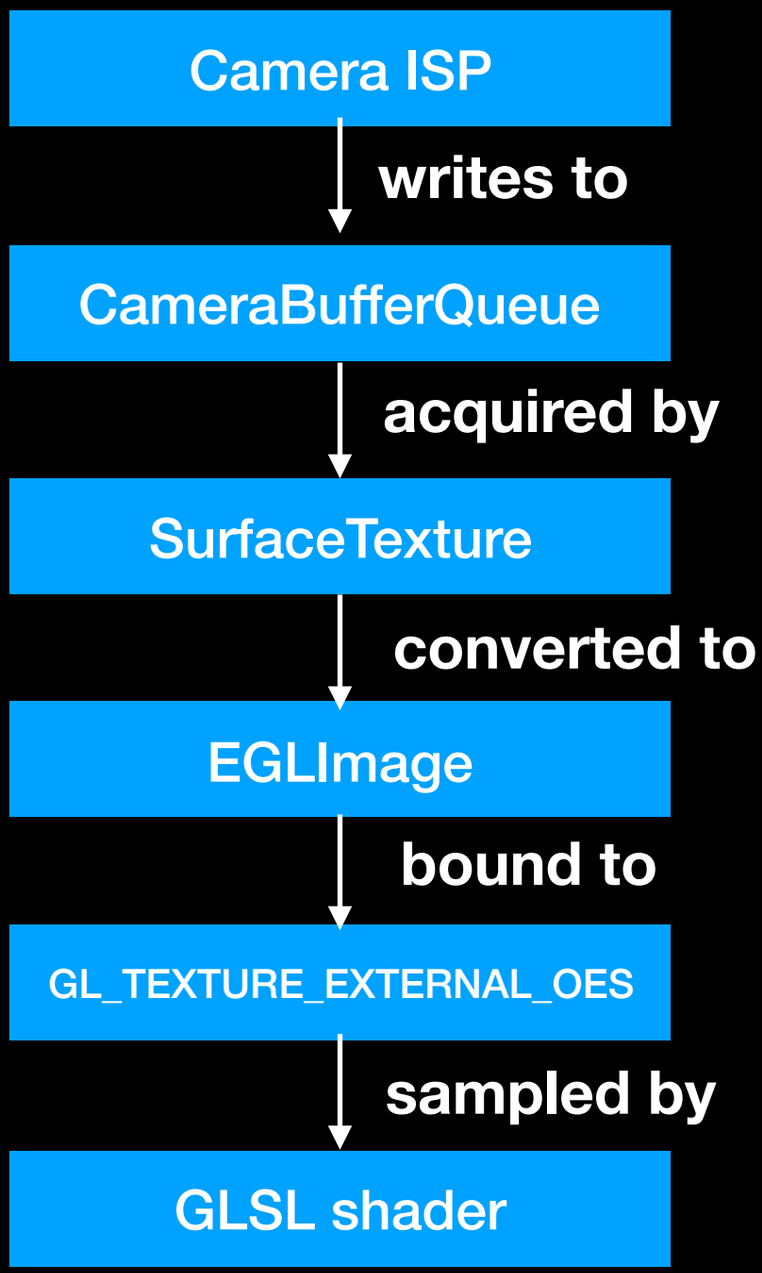
- Provides a zero copy data path for bringing data into the GPU.

- The same DRAM memory can be accessed by different entities (without copy) starting at the ISP write and ending at the GLSL sample.

gralloc

- gralloc = graphic allocator
- gralloc buffer = a hardware shareable block (between ISP, GPU, Display, Encoder, CPU) of DRAM with metadata and usage permissions / flags attached.
- gralloc is a Hardware Abstraction Layer (HAL) module.
- gralloc is responsible for allocating hardware buffers.
- gralloc is backed by kernel memory allocators (DMA-BUF heaps)

Copy Free Data Path to Shader



The Details

- The previous flowchart provides a high level overview of the data path.
- Important details are omitted from the flowchart.
- Important details are also not fully exposed in the software.
- To provide a better understanding, the entire process will now be described in greater detail.

The Details

- We begin with Surface and SurfaceTexture.
- Surface and SurfaceTexture are part of the Android graphics systems.
- => More generally, the Android framework (SDK).
- Surface and SurfaceTexture provide the interconnect / interface between the Camera and OpenGL.

Surface

- Surface is a class in the package android.view.
- i.e. import android.view.Surface
- Surface is the producer endpoint handle to BufferQueue.
- Surface is NOT a GL object, a texture, a framebuffer, or part of OpenGL.

SurfaceTexture

- SurfaceTexture is a class in the package android.graphics.
- i.e. import android.graphics.SurfaceTexture

- SurfaceTexture is a consumer endpoint handle to the BufferQueue.
- SurfaceTexture is the bridge between Android graphics and OpenGL ES.
- SurfaceTexture is the object that updates a GL texture.

BufferQueue

- Next we define BufferQueue.
- BufferQueue is the handoff mechanism between two parties: Producer & Consumer
- OpenGL employs two different BufferQueues.
- The first BufferQueue is the CameraBufferQueue.
- The second BufferQueue is the WindowBufferQueue.

BufferQueue

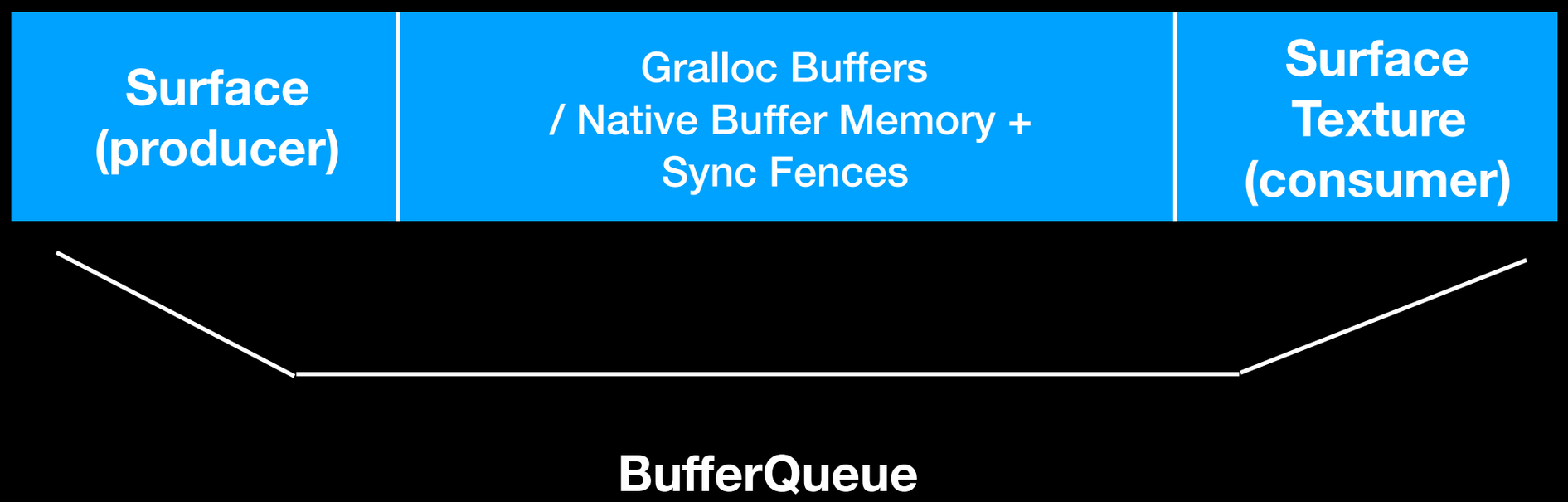
- The BufferQueue coordinates a producer and consumer by managing gralloc buffers / native buffer memory, synchronization fences, and buffer ownership.

BufferQueue

The main components of the BufferQueue are

- Gralloc Buffers / Native Buffer Memory
- Producer Endpoint
- Consumer Endpoint
- Sync Fences

High Level Mental Picture



Gralloc Buffers

- Memory allocated by the system (gralloc)
- Mappable by the GPU
- Often backed by AHardwareBuffer
- Contains Usage Flags

Producer Endpoint

- API: Camera HAL output “Surface”

Actions

- Dequeue a free gralloc buffer
- Render / Fill
- Queue for consumption

Consumer Endpoint

- API: SurfaceTexture

Actions

- Acquire a queued gralloc buffer
- Use it
- Release it back to the pool

Sync Fence

- Tells the consumer or producer when it is “OK” to read or write.
- 1) Producer (or Consumer) issues the Ownership Action of Release.
- This indicates that the work has been issued, but not necessarily completed by the hardware - i.e. NOT SIGNALED.
- 2) The Consumer (or Producer) issues an Ownership Action of Acquire.
- This indicates that the Consumer (or Producer) is waiting for the hardware completion signal.
- Once wait(F) returns SIGNALED, the Consumer (or Producer) can proceed.

Asynchronous

- At RELEASE, the hardware completion signal is NOT SIGNALED.

- Tthe hardware completion signal is asynchronous.

- i.e. The hardware completion signal could occur “nearly immediately” after RELEASE, or, incur some delay, after RELEASE.

Buffer States

- Free - gralloc buffer available for producer to dequeue
- Dequeued - currently in use by producer
- Queued - producer finished
- Acquired - currently in use by consumer
- In general, multiple gralloc buffers are created / exist in a BufferQueue to reduce stalls; multiple gralloc buffers do increase system latency

BufferQueue Timeline (Asynchronous)

Gralloc Buffer Free

Camera

Gralloc Buffer Free

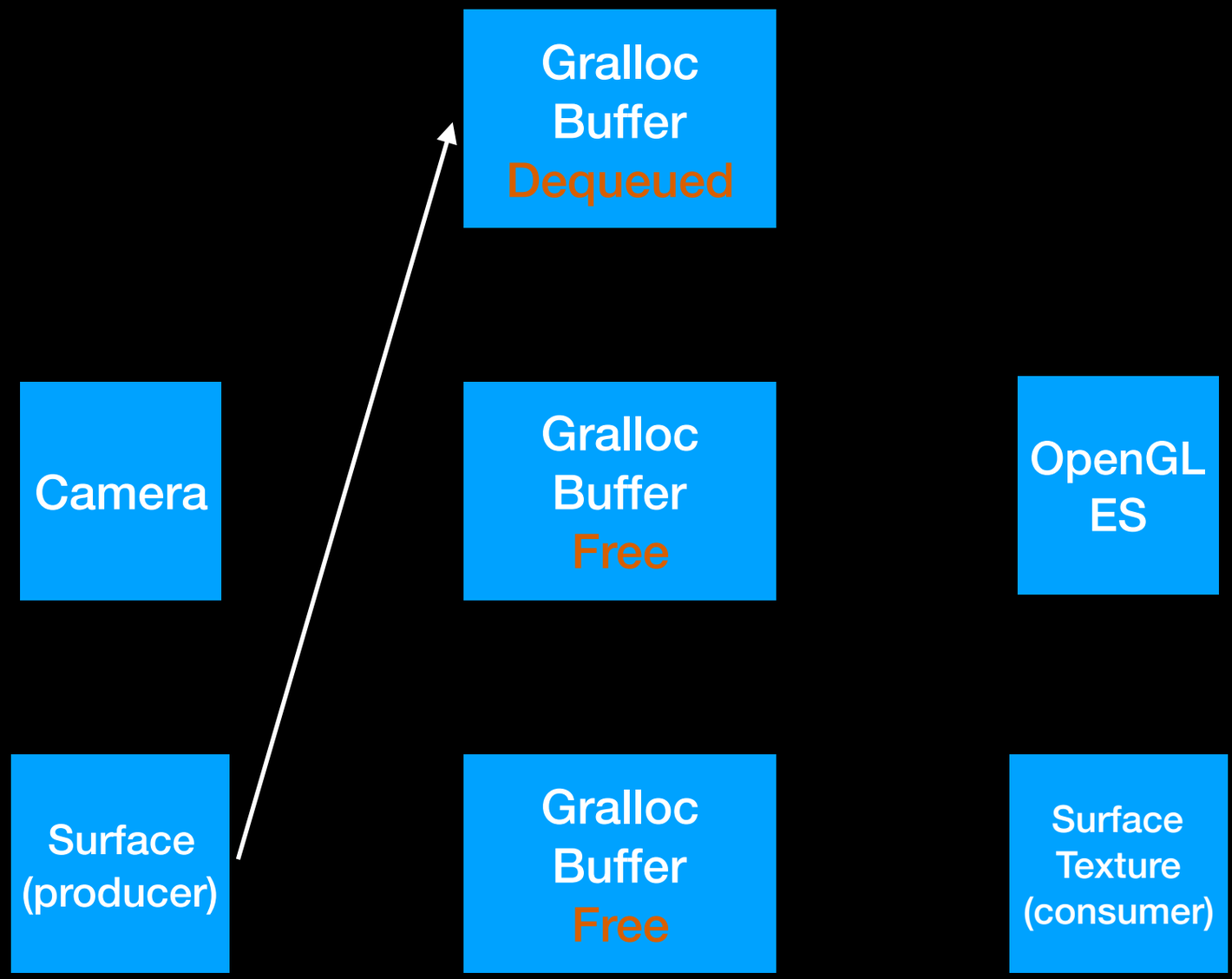
OpenGL ES

Surface (producer)

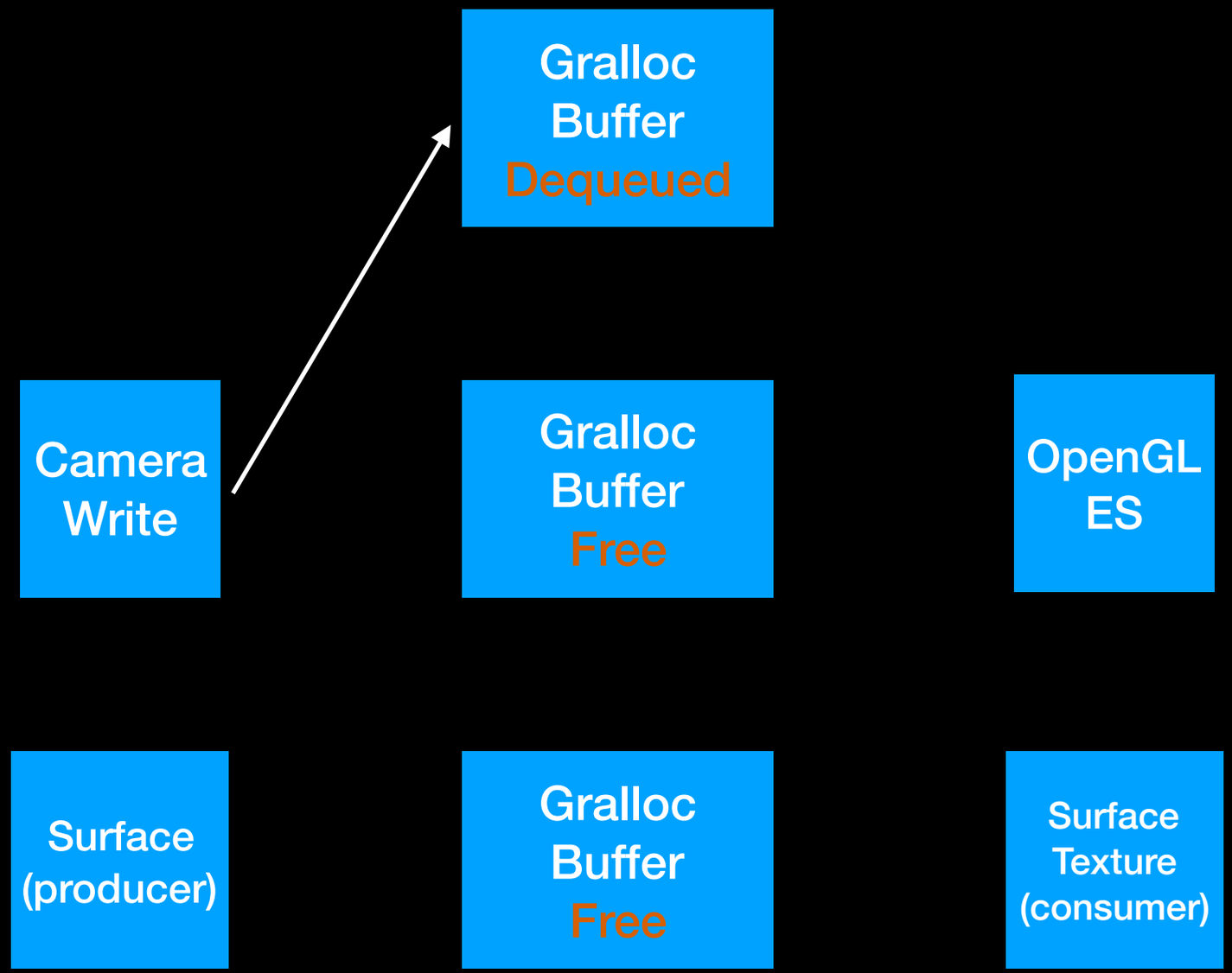
Gralloc Buffer Free

Surface Texture (consumer)

BufferQueue Timeline (Asynchronous)



BufferQueue Timeline (Asynchronous)



BufferQueue Timeline (Asynchronous)

Gralloc Buffer
Queued:Release
NOT SIGNALED

Camera

Gralloc Buffer
Free

OpenGL ES

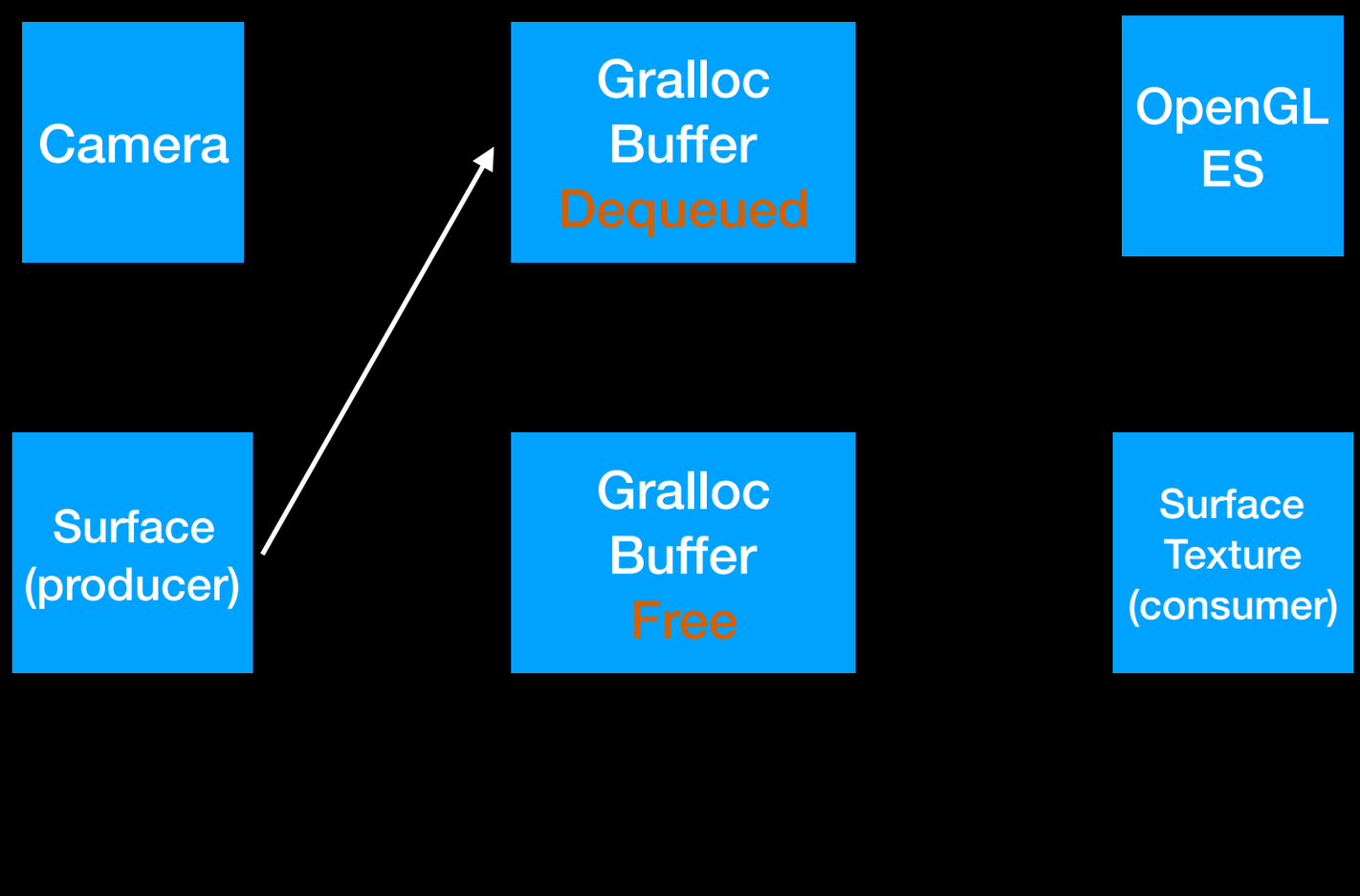
Surface (producer)

Gralloc Buffer
Free

Surface Texture (consumer)

BufferQueue Timeline (Asynchronous)

Gralloc Buffer
Queued:Release
NOT SIGNALED



BufferQueue Timeline (Asynchronous)

Gralloc Buffer
Queued:Release
NOT SIGNALED

Camera Write

Gralloc Buffer Dequeued

OpenGL ES

Surface (producer)

Gralloc Buffer Free

Surface Texture (consumer)

BufferQueue Timeline (Asynchronous)

Gralloc Buffer
Queued:Release
NOT SIGNALED

Camera

Gralloc Buffer
Queued:Release
NOT SIGNALED

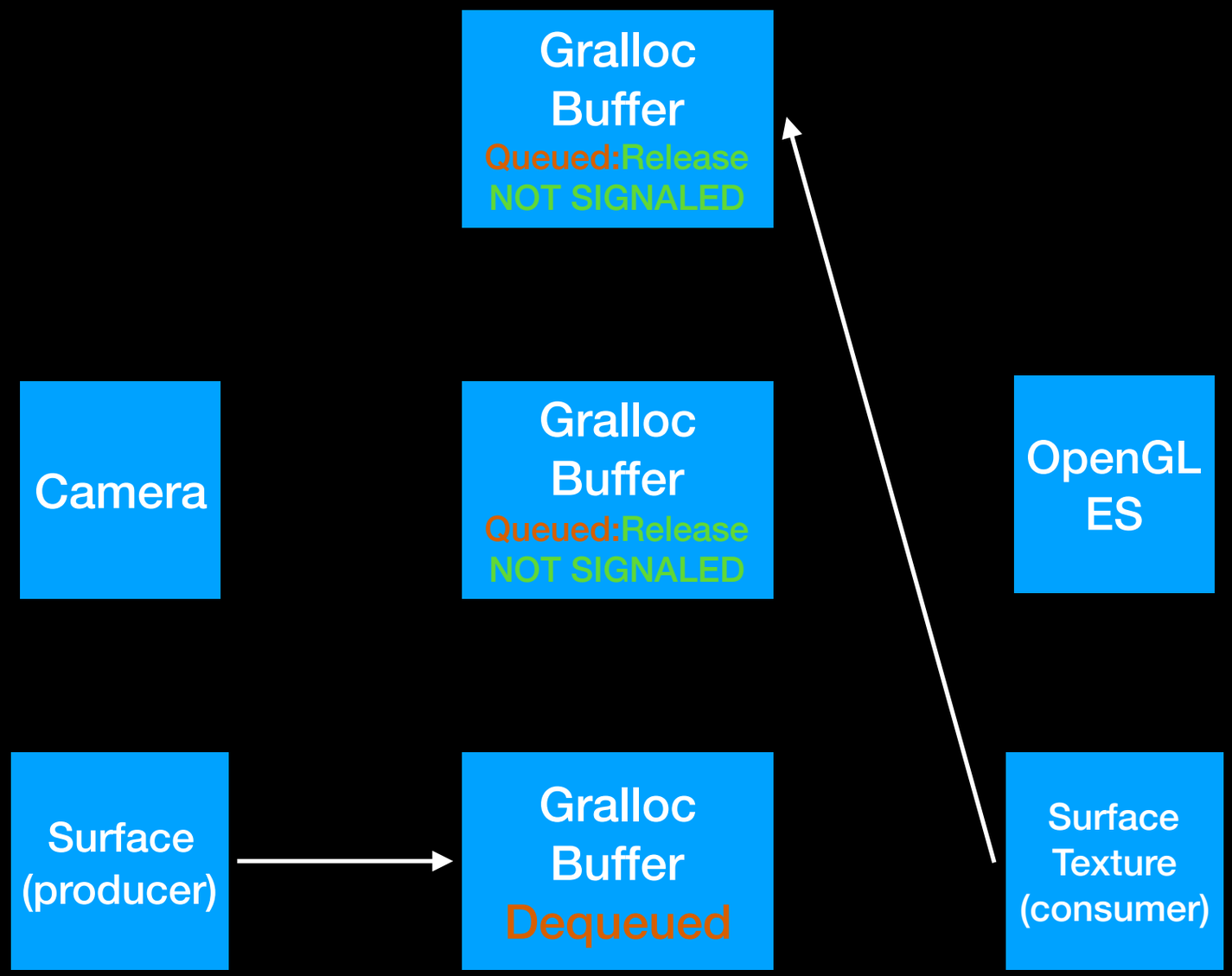
OpenGL ES

Surface (producer)

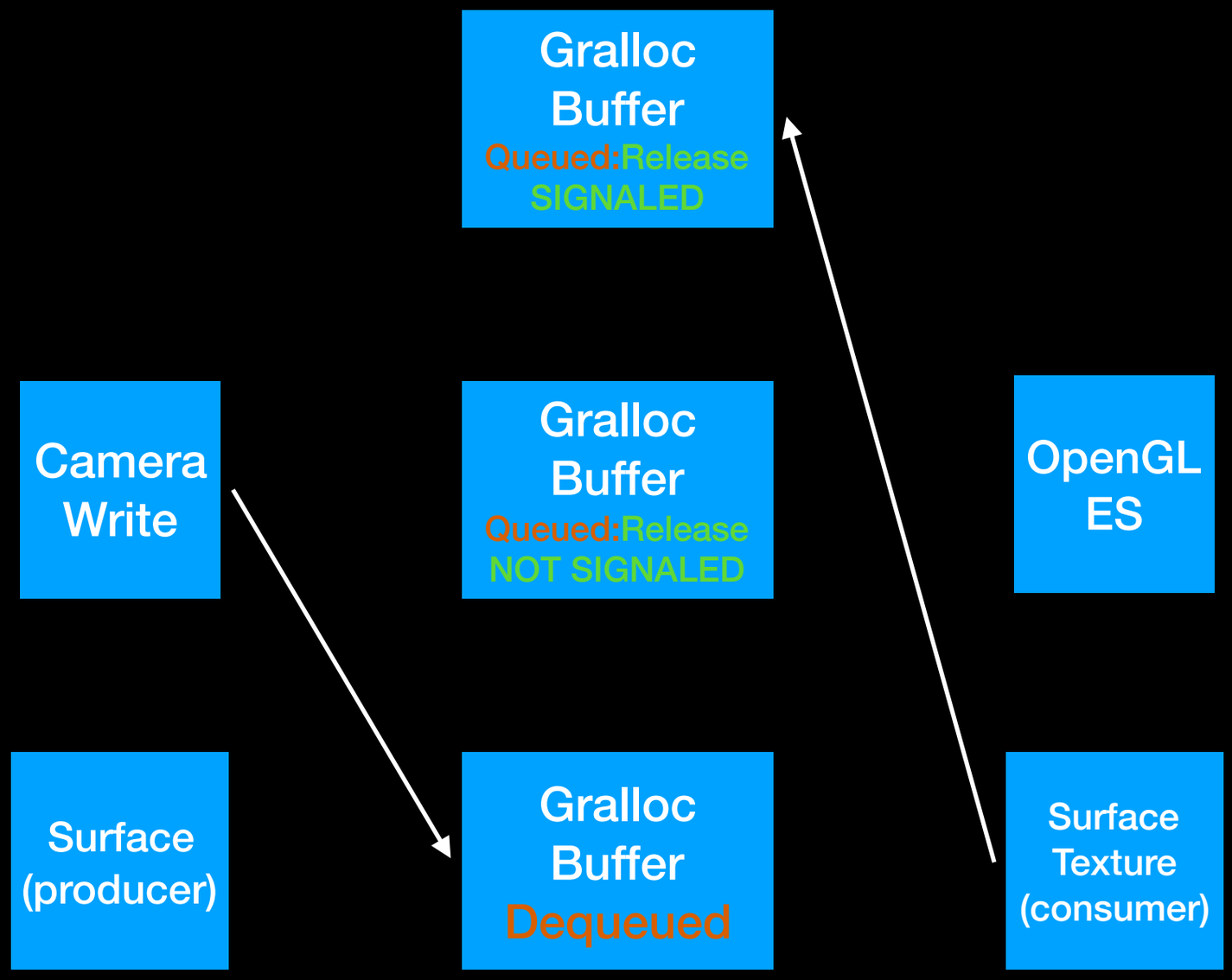
Gralloc Buffer
Free

Surface Texture (consumer)

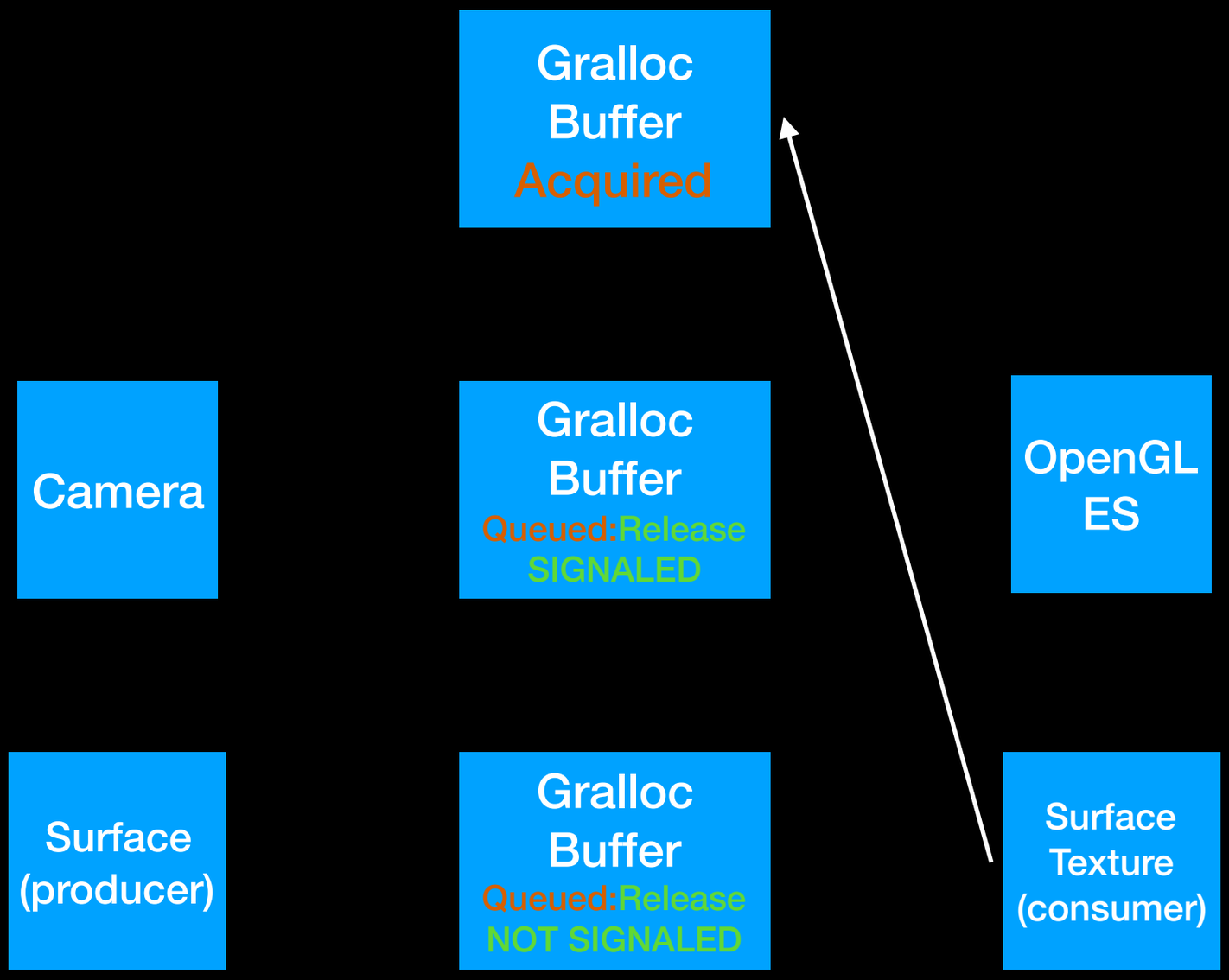
BufferQueue Timeline (Asynchronous)



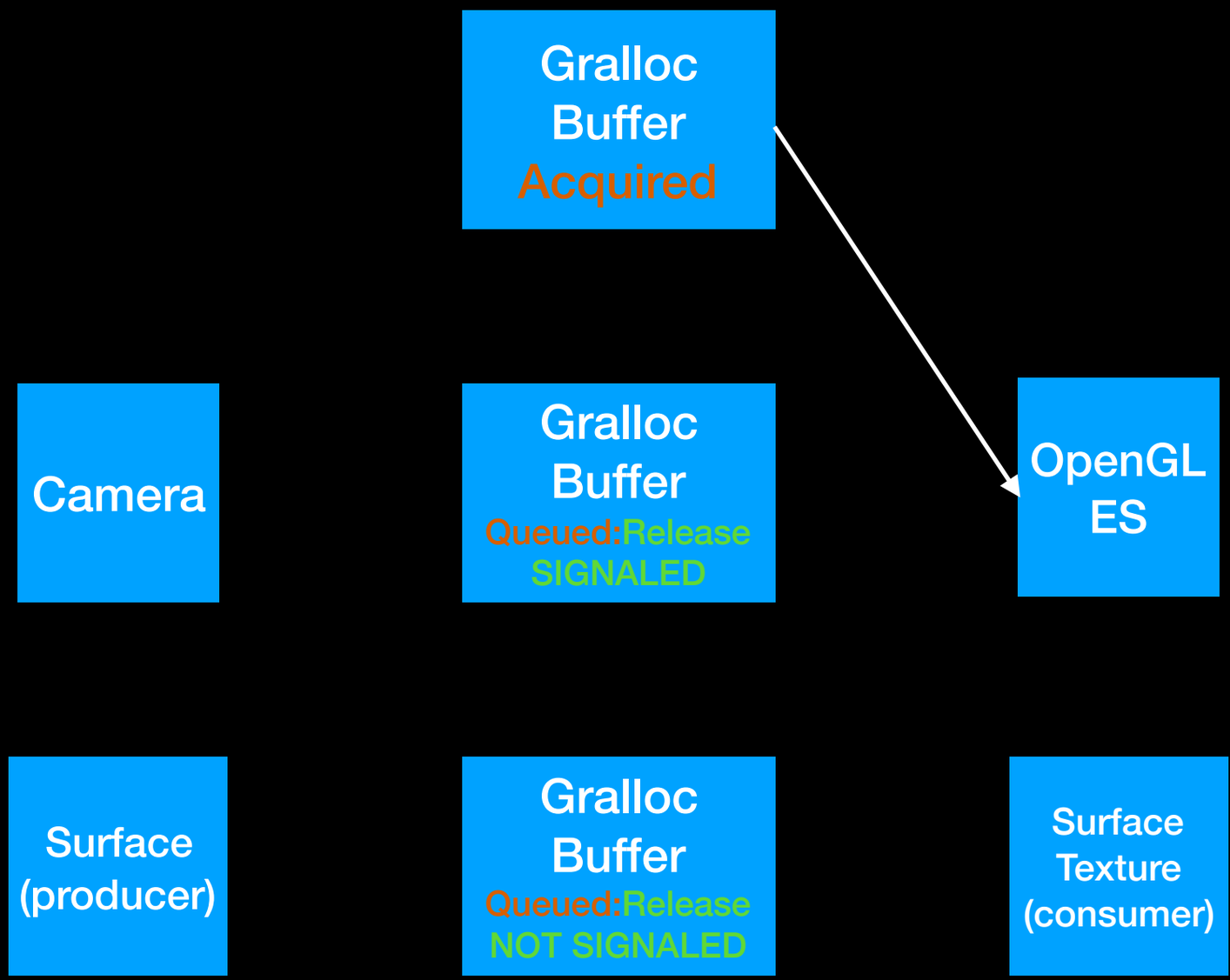
BufferQueue Timeline (Asynchronous)



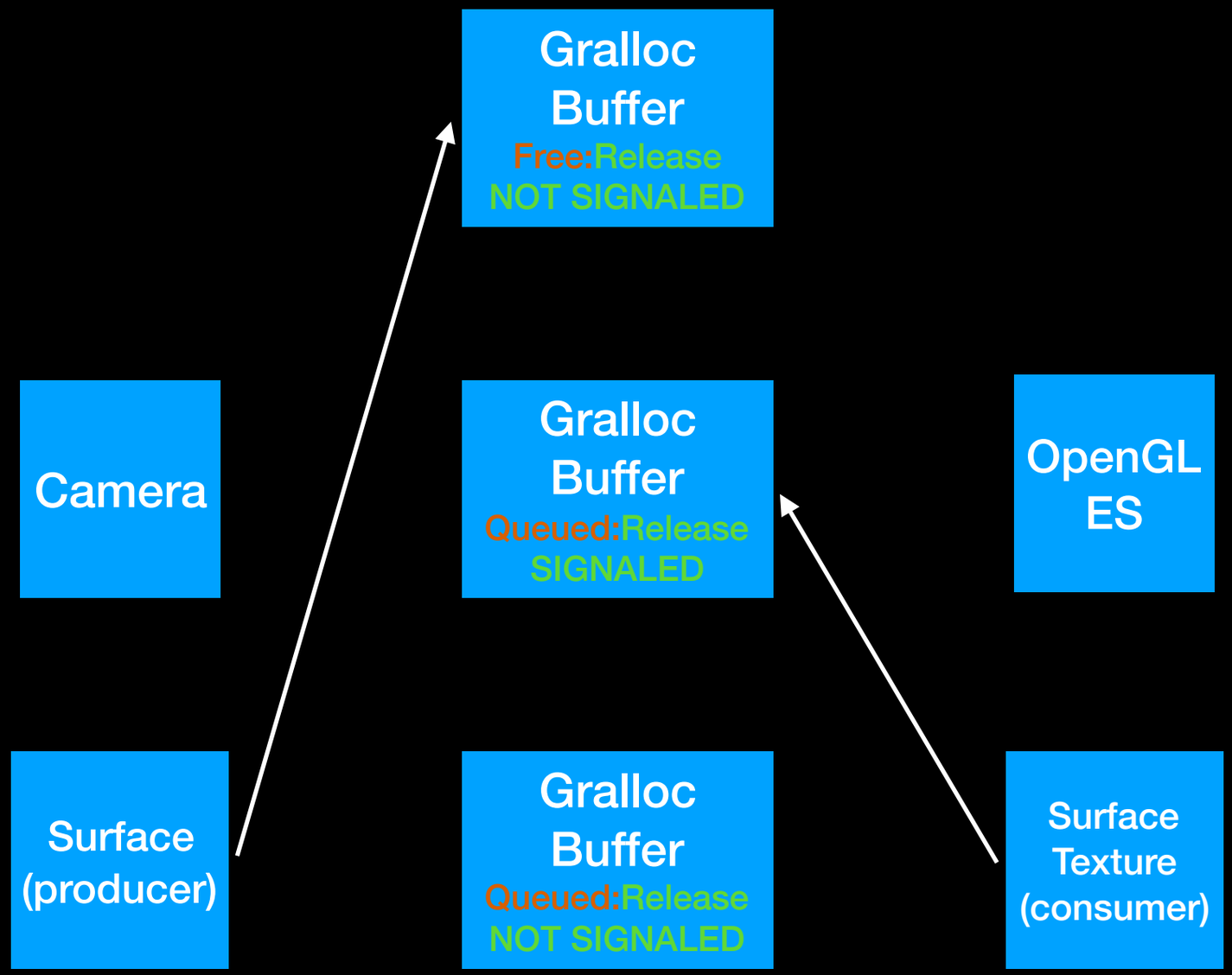
BufferQueue Timeline (Asynchronous)



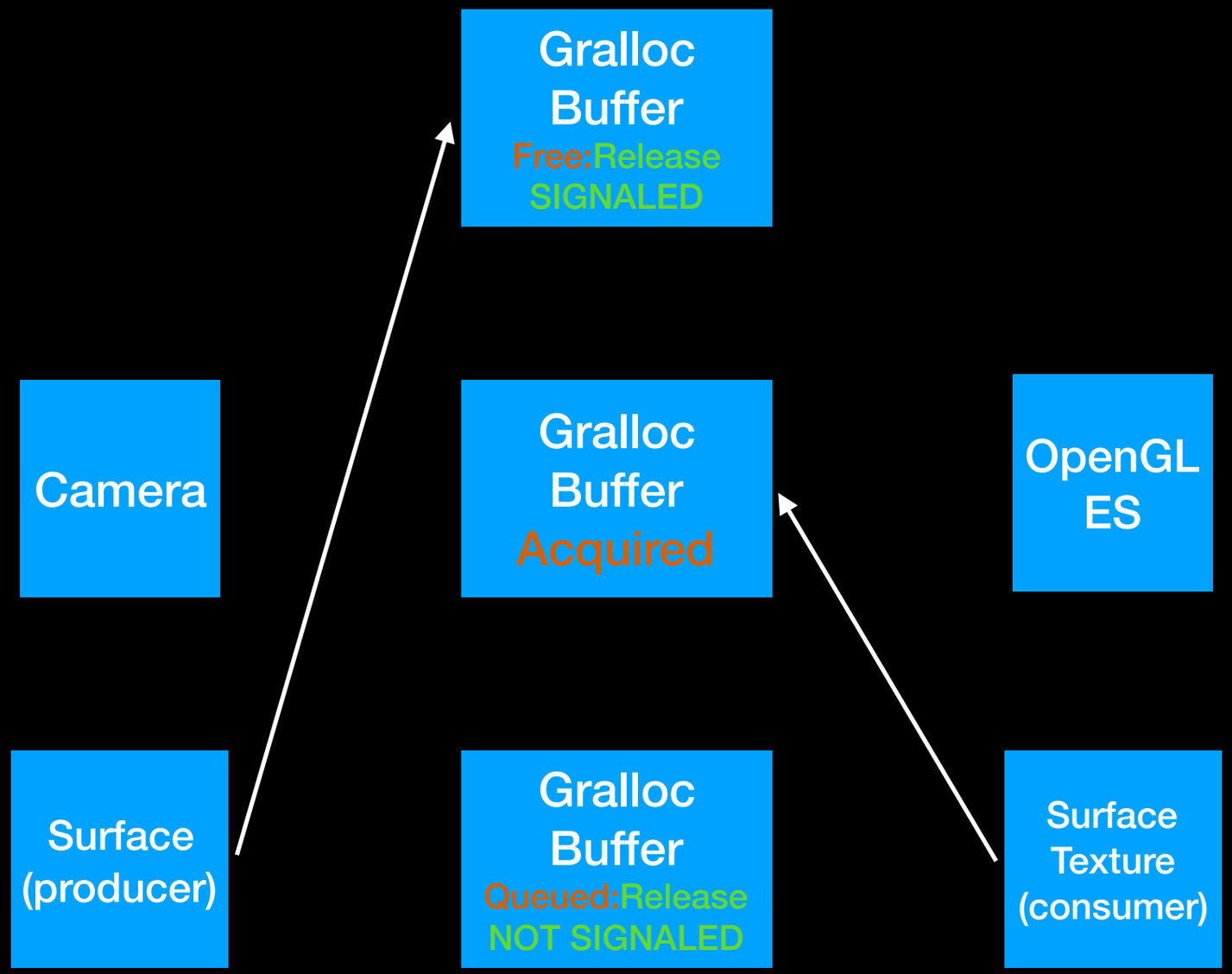
BufferQueue Timeline (Asynchronous)



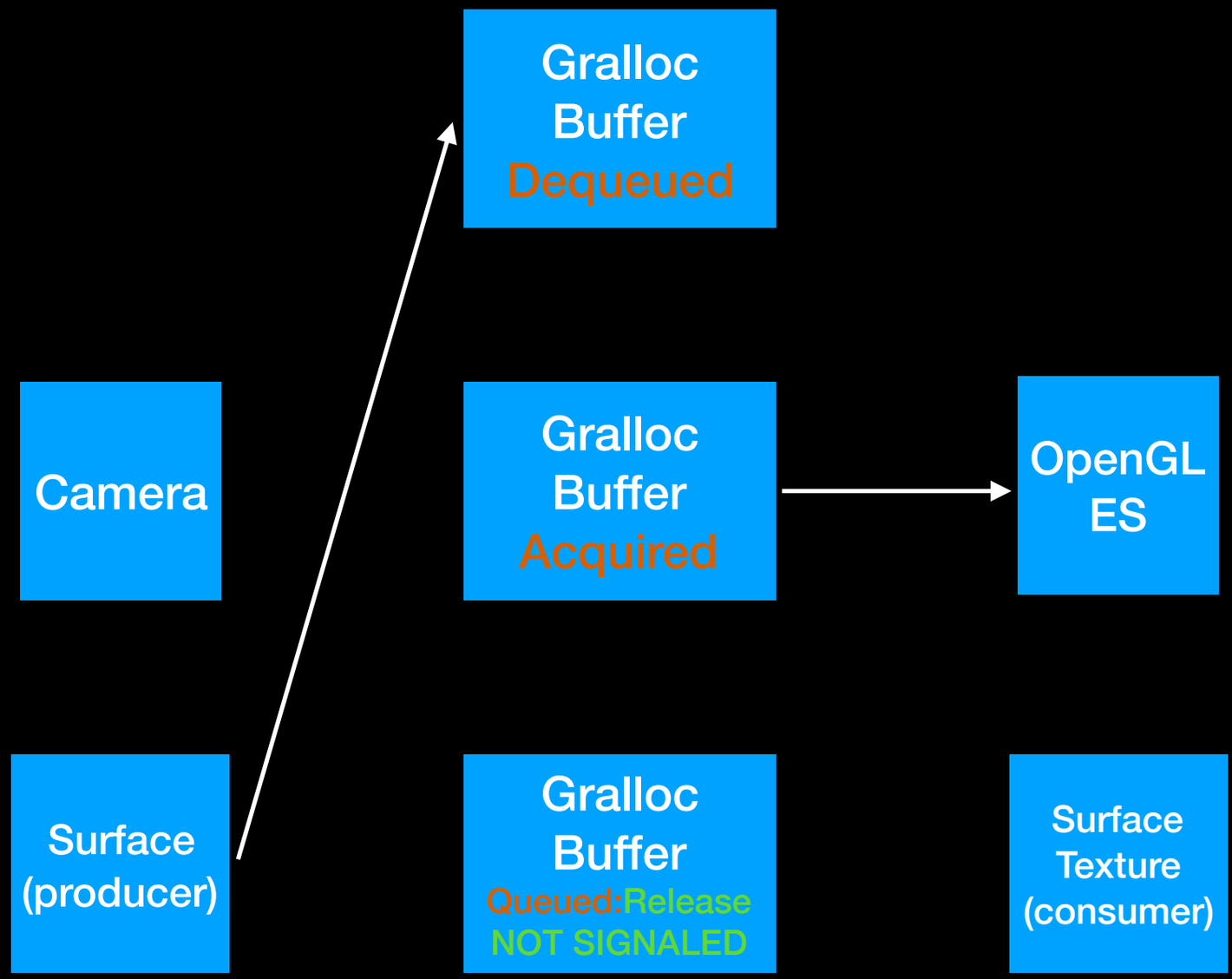
BufferQueue Timeline (Asynchronous)



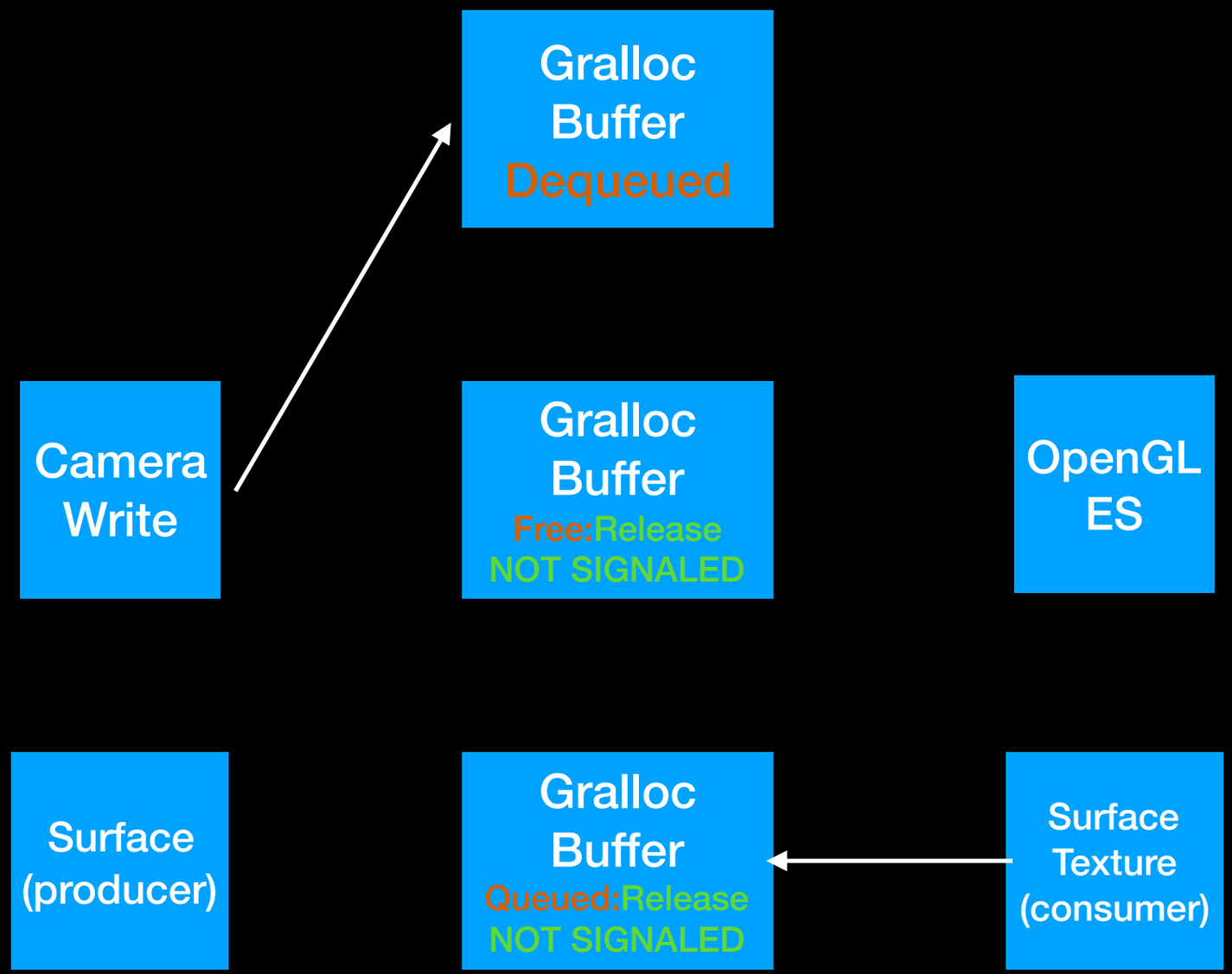
BufferQueue Timeline (Asynchronous)



BufferQueue Timeline (Asynchronous)



BufferQueue Timeline (Asynchronous)



BufferQueue Timeline (Asynchronous)

Gralloc Buffer
Queued:Release
NOT SIGNALED

Camera

Gralloc Buffer
Free:Release
NOT SIGNALED

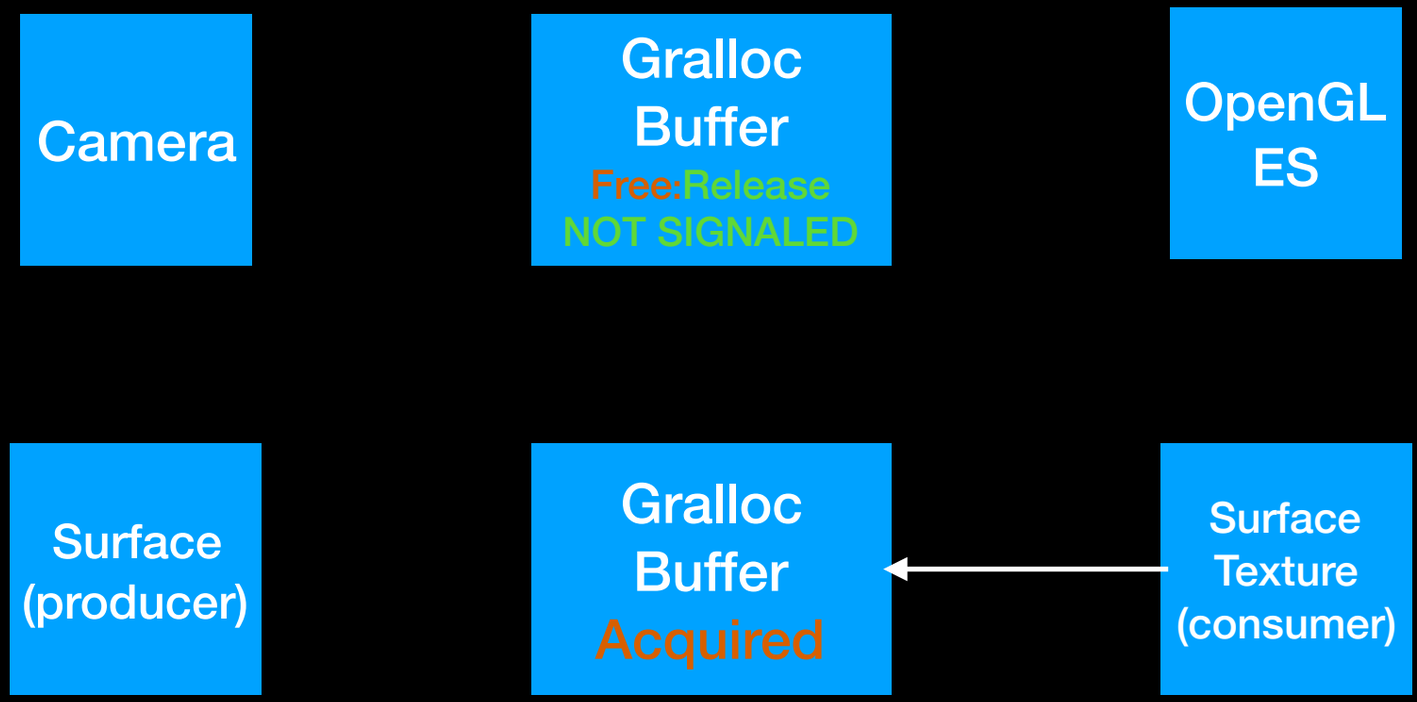
OpenGL ES

Surface (producer)

Gralloc Buffer
Queued:Release
SIGNALED

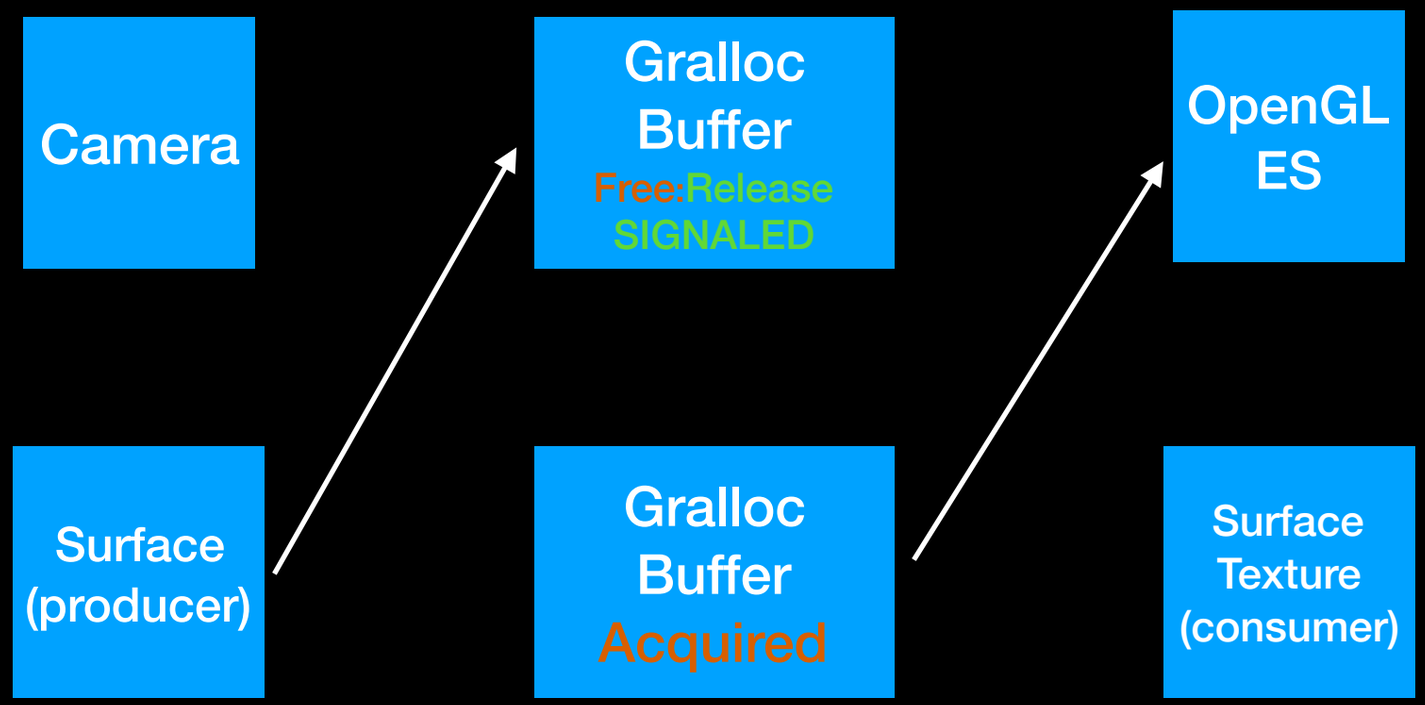
Surface Texture (consumer)

BufferQueue Timeline (Asynchronous)



BufferQueue Timeline (Asynchronous)

Gralloc
Buffer
Queued:Release
SIGNALED



BufferQueue Timeline

- etc., the process continues …
- Typically, a ring buffer configuration (consisting of 3 gralloc buffers) is deployed.
- The ring buffer configuration minimizes pipeline stalls.
- The ring buffer configuration increases memory usage and system latency.

The Details

- In our use case, the producer is associated with a Surface that receives Camera ISP frames.
- In our use case, the consumer is associated with a SurfaceTexture that interfaces with OpenGL ES.
- The producer and the consumer **manage the use of the gralloc buffers.**
- Hardware completion signals are sent, upon the completion of the task.
- The consumer and producer enforce the hardware completion signal.

The Details

- After the Camera is done issuing its writes, or OpenGL ES is done issuing its reads, a Release signal is sent.

- However, this does not mean that the task has physically completed.

- The task has physically completed, ONLY when the hardware completion signal has been sent - SIGNALED.

Specifics

- Camera HAL requests the creation of gralloc buffers for the BufferQueue, shortly after the camera is ready to write frames from the ISP.

- The ISP DMA engine can write into a gralloc buffer, when the hardware has SIGNALED completion of the consumer task for the specific gralloc buffer.

- OPENGL ES can read from a gralloc buffer, when the hardware has SIGNALED completion of the producer task for the specific gralloc buffer.

- BufferQueue manages ownership between producers and consumers.

The Flow

- We (should) now have a good understanding of the BufferQueue.

- Next, we will map the “Copy Free Data Path to Shader” block diagram slide to the source code.

The Flow

- In MainActivity, an instance of GLSurfaceView is created.
- In GLSurfaceView, a GL Renderer object is created.
- However, the GL Renderer object cannot perform any tasks, without a GL thread.
- When the instance of GLSurfaceView is passed to setContentView( ), a GL thread is created.
- Given the existence of a GL thread and the GL Renderer object, an EGL context can then be created.
- This enables the onSurfaceCreated method to be called, resulting in the creation of a GL context.

The Flow

- Once the GL context has been created, a texture ID and an OES texture can be defined.

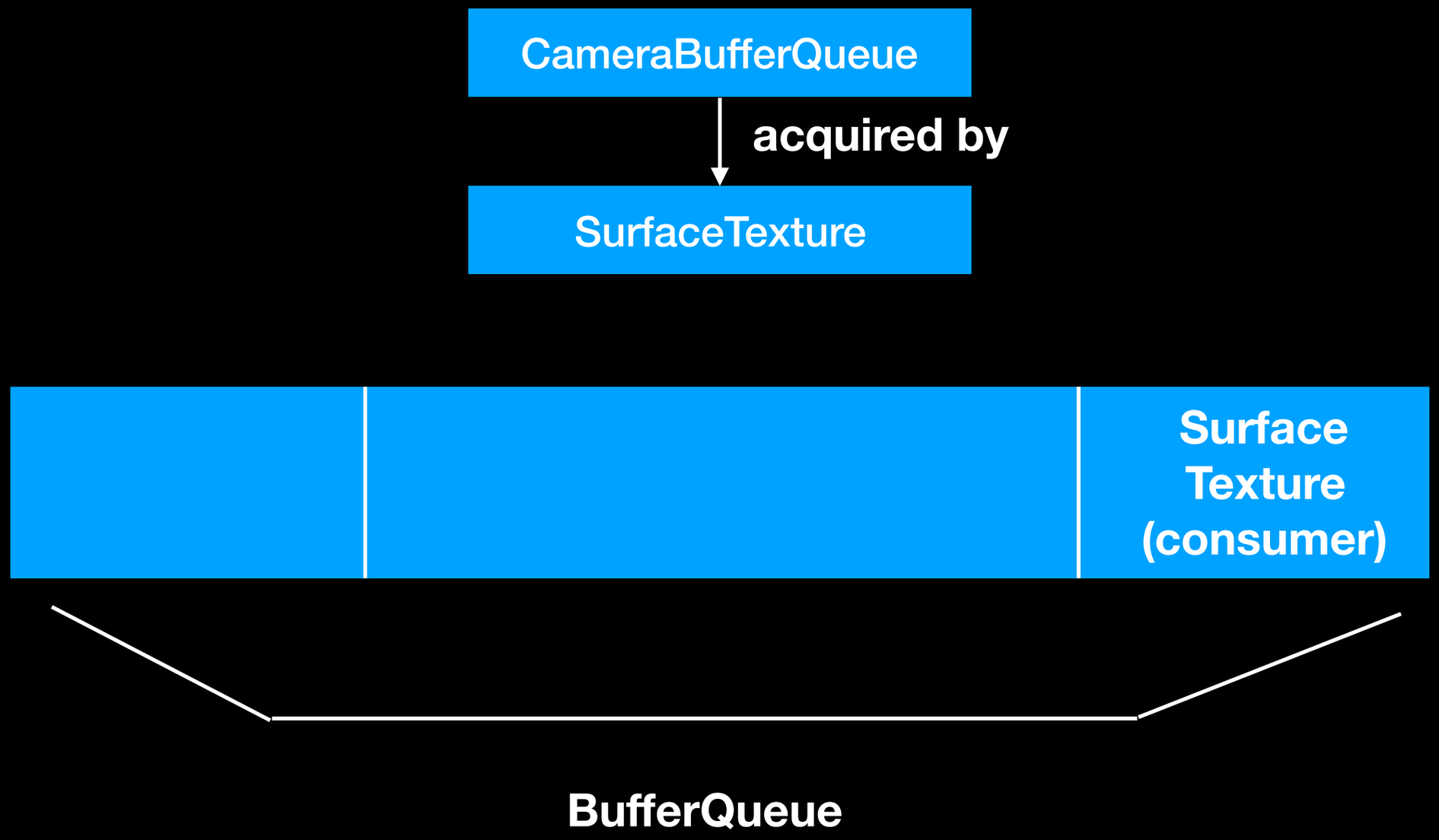
- 1) Specifically, the BufferQueue and the consumer endpoint are now simultaneously created: surfaceTexture = SurfaceTexture(textureID).

- In GL Renderer, we also define the variable: private var surfaceTextureReadyCallback: ((SurfaceTexture) ->Unit)? = null

- This variable stores the function passed to it from the callback in MainActivity.

- The function in MainActivity is passed when surfaceTextureReadyCallback?.invoke(surfaceTexture) is executed in onSurfaceCreated.

Copy Free Data Path to Shader (1)



The Flow

- Contained in this function, is an instance of SurfaceTexture created in MainActivity.

- The physical surfaceTexture created in the GL Renderer is then linked with this instance of SurfaceTexture in MainActivity.

- 2) The producer endpoint in BufferQueue is then generated using: surface = Surface(surfaceTexture)

- 3) Camera frames can then be written to the producer endpoint, surface.

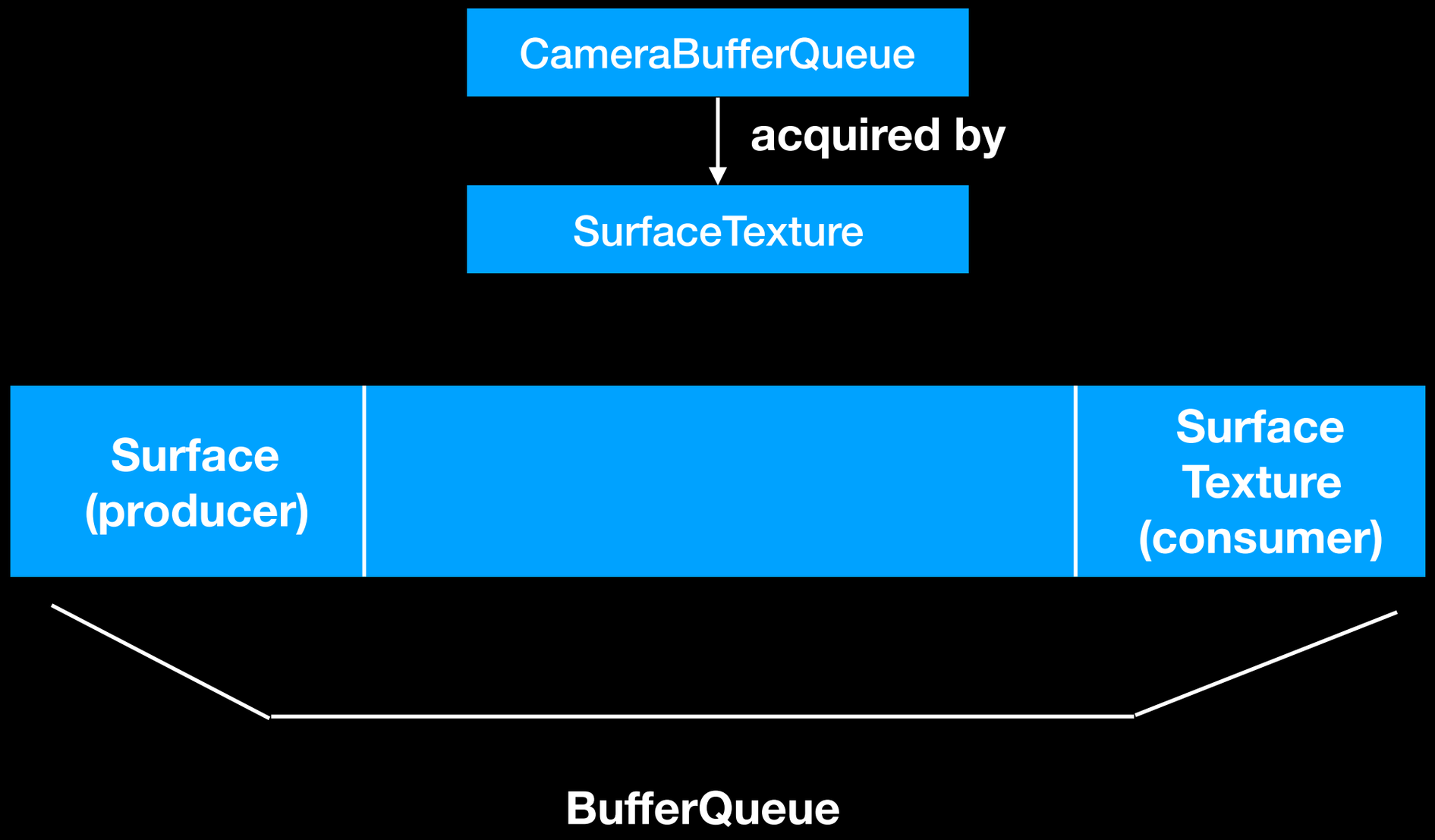
- At the same time, we set:

```
surfaceTexture.setOnFrameAvailableListener {
    glSurfaceView.requestRender()
}
```

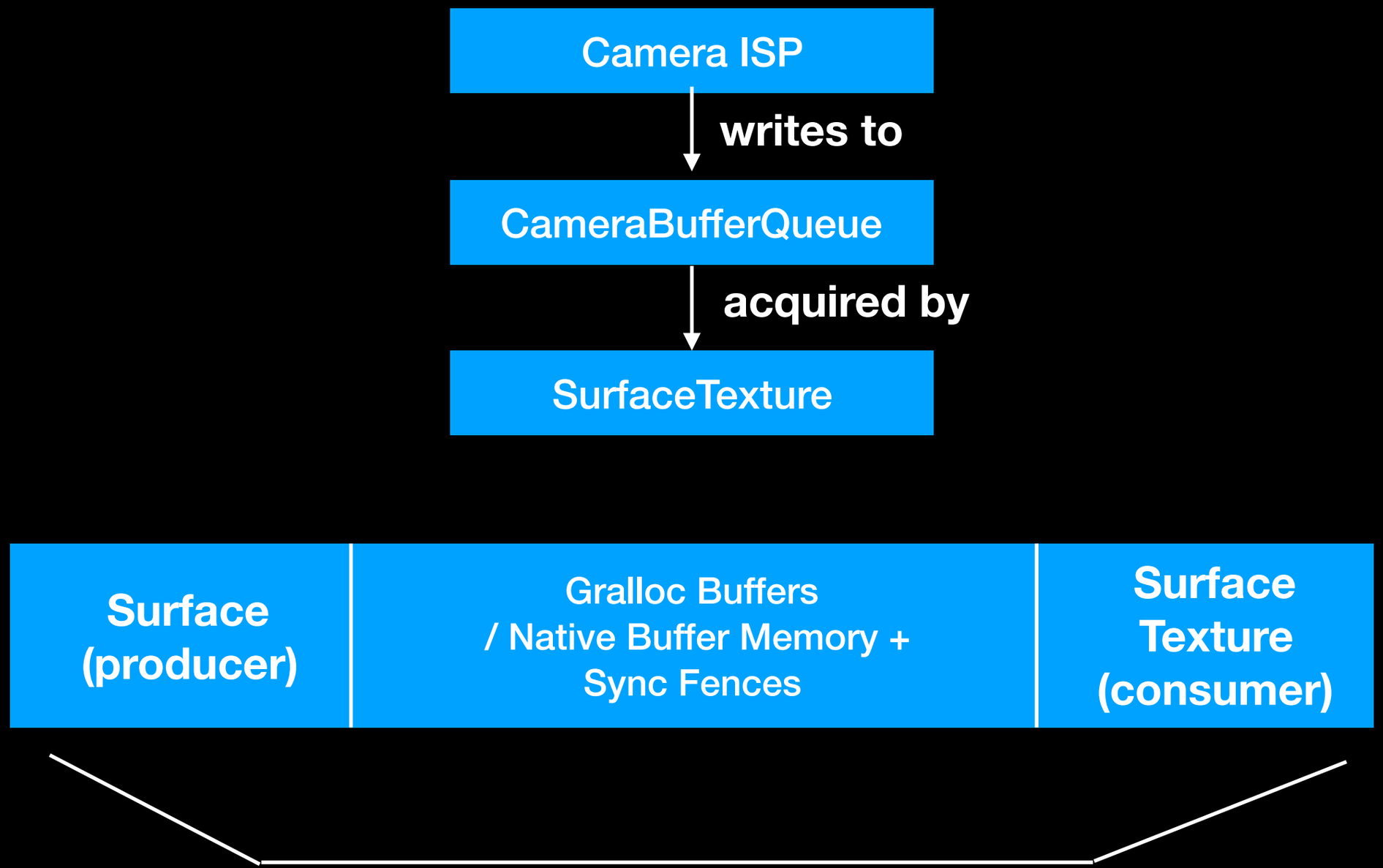
Regarding (3)

- Camera HAL requests buffers from gralloc.
- In software, this occurs at:
  cameraProvider.bindToLifecycle(this, cameraSelector, preview)
- The ISP DMA engine then writes to the buffer
- The BufferQueue manages ownership between producers and consumers

Copy Free Data Path to Shader (2)



Copy Free Data Path to Shader (3)



The Flow

- When the camera has written a new frame to the BufferQueue, the listener is informed.

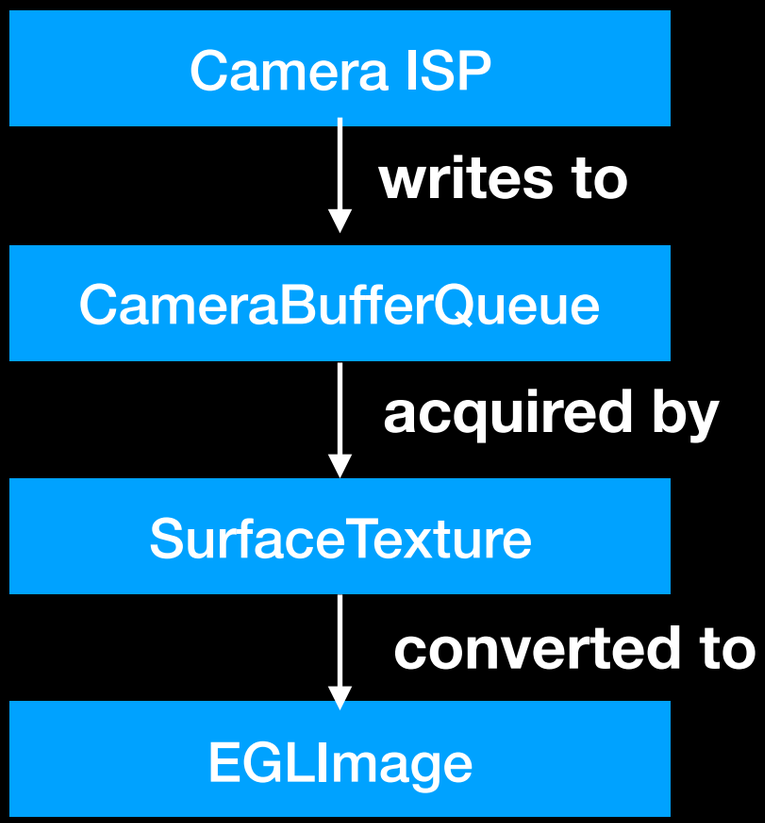
- The listener calls glSurfaceView.requestRender( ) which calls onDrawFrame( ).

- 4) onDrawFrame( ) executes updateTexImage( ).

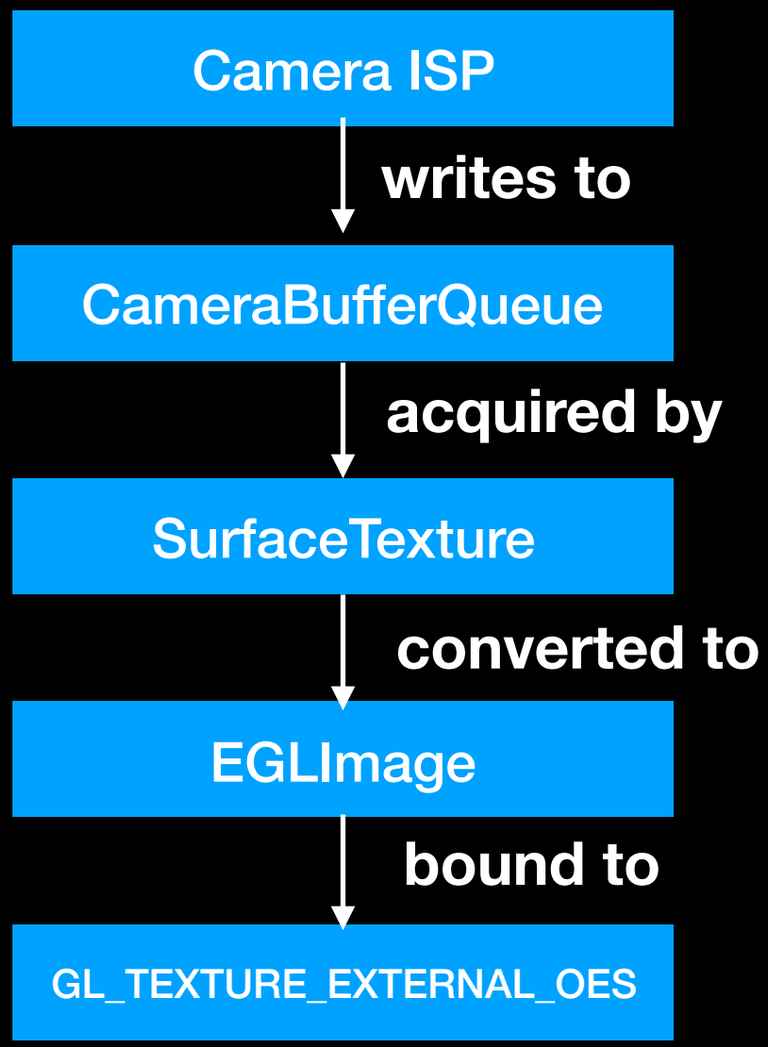
- 5) The newest queued gralloc buffer is then bound for sampling.

- 6) GLSL samples the gralloc buffer using SamplerExternalOES.

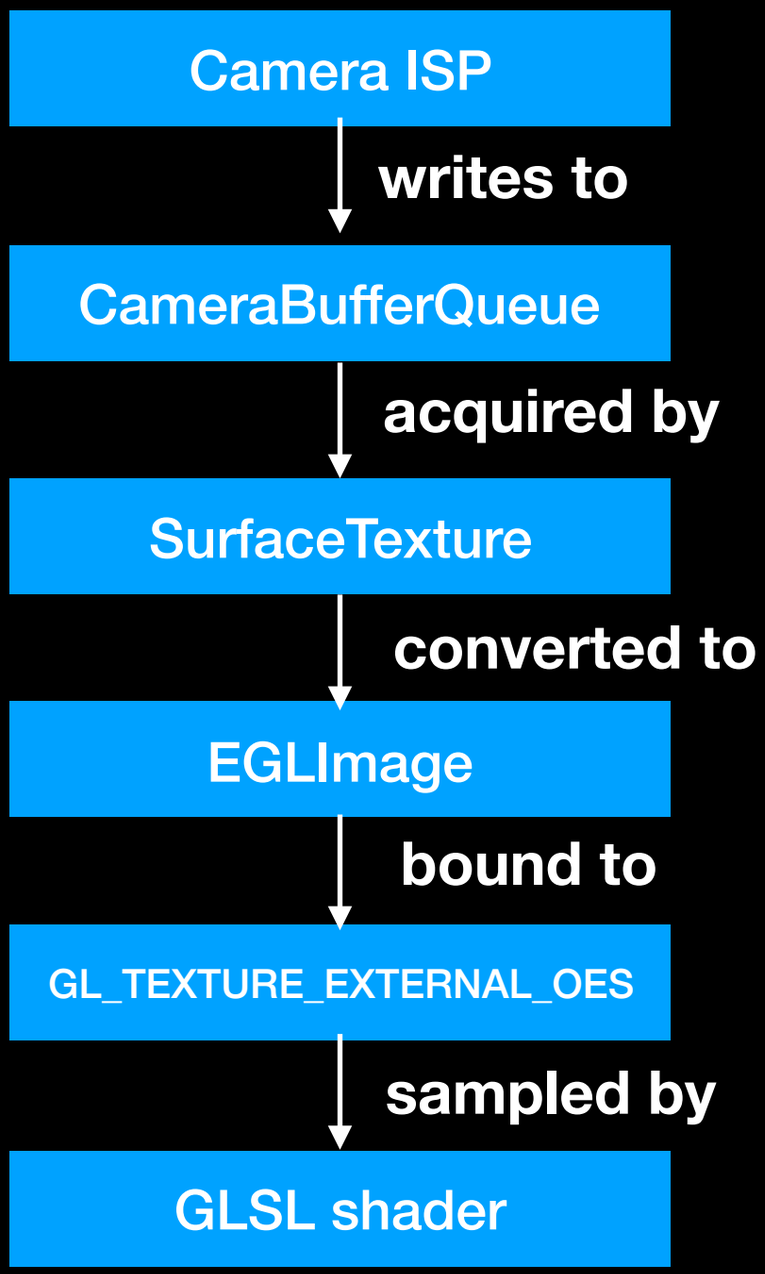
Copy Free Data Path to Shader (4)



Copy Free Data Path to Shader (5)



Copy Free Data Path to Shader (6)



The Flow

MainActivity (UI Thread)
|
glSurfaceView = GLSurfaceView(this)
setContentView(glSurfaceView)
|
GLSurfaceView attaches to window
|
renderer = GLRenderer(this)
setRenderer(renderer)
|
GL Renderer thread starts
|
EGL context created
|
MyGLRenderer.onSurfaceCreated()
|
GL Context created
|
Create OES Texture
SurfaceTexture(textureID)
|
surfaceTextureReadyCallback.invoke(surfaceTexture)
|
MainActivity.startCamera()
|
Camera writes frames via Surface(surfaceTexture)
|
OnFrameAvailableListener
|
requestRenderer()
|
onDrawFrame()
|
updateTexImage()
glDrawArrays

Details After Consumer Endpoint

- SurfaceTexture provides access to the Gralloc Buffers / Native Buffer Memory written to, by the Camera ISP.

- In practice, GLSL shaders expect input similar to that created by glTexImage2D( ) - RGB, etc.

- However, the ISP may have written YUV data to the Gralloc Buffers / Native Buffer Memory.

- EGLImage provides a wrapper around the Gralloc Buffers / Native Buffer Memory.

- This wrapper creates an OpenGL Texture Memory view of the existing Gralloc Buffers / Native Buffer Memory.

Details After Consumer Endpoint

EGLImage = {
Pointer to native buffer
Pixel format
Plane layout
Stride
}

Pointer to native buffer can include: NV12, NV21, YUV420, etc.

GLSL shader wants: RGBA8, RGB8, R8, etc.

Details After Consumer Endpoint

Gralloc Buffers / Native Buffer Memory

OpenGL Texture Memory

We need to ATTACH the OpenGL Texture Memory to the Gralloc Buffers.

Details After Consumer Endpoint

We need to ATTACH the OpenGL Texture Memory to the Gralloc Buffers.

EGLImage performs this attachment.

EGLImage says “the information for the OpenGL Texture Memory lives in the Gralloc Buffers”.

Details After Consumer Endpoint

- GL_TEXTURE_EXTERNAL_OES defines the texture object type in OpenGL ES, telling OpenGL ES how the object is stored.

- i.e. glBindTexture(GL_TEXTURE_EXTERNAL_OES, textureId)

- i.e. GL_TEXTURE_EXTERNAL_OES says that the texture will be backed by an external image (= the texture is external).

- GL_TEXTURE_EXTERNAL_OES (= the texture is external) tells the driver to 1) not expect glTexImage2D( ), 2) that storage is provided by another API, and 3) that the sampling rules will be different.

- Finally, samplerExternalOES performs the “under the hood” conversion to bring the external image into the GLSL shader as RGBA8, RGB8, etc.

Sampling in the GLSL Shader

- #version 310 es
- #extension GL_OES_EGL_image_external_essl3 : require
- uniform samplerExternalOES **textureSampler**;
- vec4 color = texture(**textureSampler**, texCoord);

OpenGL ES Connects the Texture Memory with the Sampler

We establish the “hook” between the OpenGL texture memory and the GLSL shader sampling with:

```
dTextureUniformHandle =
GLES31.glGetUniformLocation(mShaderProgram,
"textureSampler")
GLES31.glActiveTexture(GLES31.GL_TEXTURE10)
GLES31.glBindTexture(GLES11Ext.GL_TEXTURE_EXTERNAL_OES,
textureSurface)
GLES31.glUniform1i(dTextureUniformHandle, 10)
```

Important Code Constructs

```
//Create OES Texture
int[] tex = new int[1];
glGenTextures(1, tex, 0);
glBindTexture(GLES11Ext.GL_TEXTURE_EXTERNAL_OES, tex[0]);

//Create SurfaceTexture in GL Renderer
SurfaceTexture surfaceTexture = new SurfaceTexture(tex[0]);

//Create Surface from SurfaceTexture in MainActivity
Surface surface = new Surface(SurfaceTexture);

//On each frame
surfaceTexture.updateTexImage()

//In GLSL Shader
#extension GL_OES_EGL_image_external : require
uniform samplerExternalOES cameraTex
```

•

Comparison

Copy Path

glTexImage2D( )

GPU Private Memory

Sampler2D

No Copy Path

GraphicsBuffer (gralloc)

EGLImage

GL_TEXTURE_ EXTERNAL_OES

SamplerExternalOES

One Final Subtlety

- Following the creation of a GL Thread, we generated the texture ID for GPU Memory in onSurfaceCreated.
- This texture ID is then associated with a gralloc buffer in the BufferQueue.
- However, only a single texture ID was generated for the 3 gralloc buffers.
- Will this work / how does this work?

One Final Subtlety

Gralloc Buffer Acquired

glGenTextures(1, textureSurface, 0)
glBindTexture(GL_TEXTURE_EXTERNAL_OES, textureSurface[0])

Camera

Gralloc Buffer Queued:Release NOT SIGNALED

OpenGL ES

Surface (producer)

Gralloc Buffer Dequeued

Surface Texture (consumer)

gralloc

Every gralloc buffer

- Lives in DRAM
- Has usage flags (Acquire, Release)
- Has a handle (Used by EGLImage)

Native Buffer Handle

An OS description of the gralloc allocation

- DMA-BUF file descriptors
- Width / Height
- Pixel format
- Stride / Row Pitch
- Plane Layout
- Usage Flags

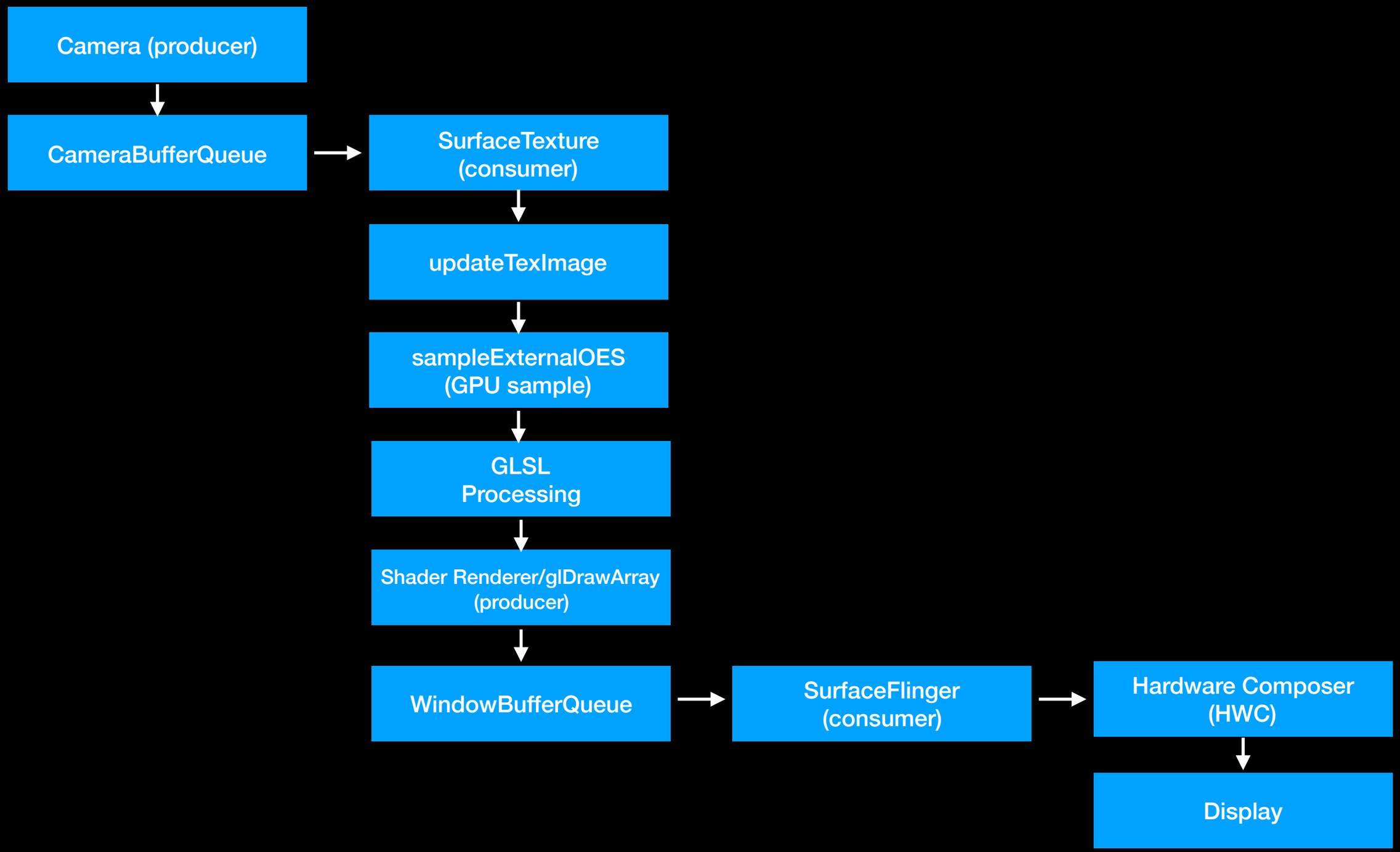
One Final Subtlety

- Every EGLImage is created from a native buffer handle associated with a specific gralloc buffer.
- (The EGLImage is a wrapper for the native buffer handle.)
- Every time updateTexImage is called, the texture object / texture ID is rebound to a different EGLImage.

GPU Processing

- The frames residing in the gralloc buffers are now available for consumption by the GLSL shader.
- In the entire input chain, zero copies have occurred.
- Once a frame has been processed by the GPU, the process of writing the frame out, begins with the issuance of glDrawArray.

End to End Pipeline



BufferQueues

CameraBufferQueue

- Producer: Camera
- Consumer: SurfaceTexture

WindowBufferQueue Example

- Producer: OpenGL ES
- Consumer: SurfaceFlinger (GPU Compositor)

To The Display

- The WindowBufferQueue operates in a similar fashion to the CameraBufferQueue.
- i.e. There are gralloc buffers, a producer endpoint, a consumer endpoint, and sync fences.
- i.e. The same mechanism is used, with the same constructs and definitions.
- glDrawArray submits the work (as producer) to generate the desired frame output.

To The Display

- However, until the hardware completion signal is issued (SIGNALED), the gralloc buffer cannot be accessed / read by SurfaceFlinger.

- Similarly, once SurfaceFlinger embarks on reading a frame, the frame cannot be written to, until a hardware completion signal is issued (SIGNALED).

- In practice, two gralloc buffers are employed in a ping pong fashion (for glDrawArray) in the WindowBufferQueue.

- Ping ponging allows one gralloc buffer to be written to by glDrawArray, while the gralloc buffer (from the previous frame) is being read from.

To The Display

- SurfaceFlinger runs in the CPU.
- SurfaceFlinger acts as a director that chooses whether the processed frame (and overlays) are sent to the Hardware Composer or “a secondary GPU composer”.
- The composer renders the frame and overlays into a single frame buffer.
- This frame buffer is then sent to the Display for scan out.

Summary

- That’s it!
- The devil IS in the details.